

accurately recall the identity or order of words it has recently seen (Adi et al., 2016). This is an obvious limitation of encoding a variable length sequence into a fixed dimensionality vector.

In our task, where we may want a pointer window where the length L is in the hundreds, accurately modeling all of this information within the RNN hidden state is impractical. The position of specific words is also a vital feature as relevant words eventually fall out of the pointer component’s window. To correctly model this would require the RNN hidden state to store both the identity and position of each word in the pointer window. This is far beyond what the fixed dimensionality hidden state of an RNN is able to accurately capture.

For this reason, we integrate the gating function directly into the pointer network by use of the sentinel. The decision to back-off to the softmax vocabulary is then informed by both the query q , generated using the RNN hidden state h_{N-1} , and from the contents of the hidden states in the pointer window itself. This allows the model to accurately query what hidden states are contained in the pointer window and avoid having to maintain state for when a word may have fallen out of the pointer window.

2.6. Pointer Sentinel Loss Function

We minimize the cross-entropy loss of $-\sum_j y_{ij} \log p(y_{ij}|x_i)$, where $\hat{y}_i$ is a one hot encoding of the correct output. During training, as $\hat{y}_i$ is one hot, only a single mixed probability $p(y_{ij})$ must be computed for calculating the loss. This can result in a far more efficient GPU implementation. At prediction time, when we want all values for $p(y_i|x_i)$, a maximum of L word probabilities must be mixed, as there is a maximum of L unique words in the pointer window of length L . This mixing can occur on the CPU where random access indexing is more efficient than the GPU.

Following the pointer sum attention network, the aim is to place probability mass from the attention mechanism on the correct output $\hat{y}_i$ if it exists in the input. In the case of our mixture model the pointer loss instead becomes:

$$-\log \left(g + \sum_{i \in I(y,x)} a_i \right), \quad (9)$$

where $I(y, x)$ results in all positions of the correct output y in the input x . The gate g may be assigned all probability mass if, for instance, the correct output $\hat{y}_i$ exists only in the softmax-RNN vocabulary. Furthermore, there is no penalty if the model places the entire probability mass, on any of the instances of the correct word in the input window. If the pointer component places the entirety of the probability mass on the gate g , the pointer network incurs

no penalty and the loss is entirely determined by the loss of the softmax-RNN component.

2.7. Parameters and Computation Time

The pointer sentinel-LSTM mixture model results in a relatively minor increase in parameters and computation time, especially when compared to the size of the models required to achieve similar performance using standard LSTM models.

The only two additional parameters required by the model are those required for computing q , specifically $W \in \mathbb{R}^{H \times H}$ and $b \in \mathbb{R}^H$, and the sentinel vector embedding, $s \in \mathbb{R}^H$. This is independent of the depth of the RNN as the the pointer component only interacts with the output of the final RNN layer. The additional $H^2 + 2H$ parameters are minor compared to a single LSTM layer’s $8H^2 + 4H$ parameters. Most state of the art models also require multiple LSTM layers.

In terms of additional computation, a pointer sentinel-LSTM of window size L only requires computing the query q (a linear layer with tanh activation), a total of L parallelizable inner product calculations, and the attention scores for the L resulting scalars via the softmax function.

3. Related Work

Considerable research has been dedicated to the task of language modeling, from traditional machine learning techniques such as n-grams to neural sequence models in deep learning.

Mixture models composed of various knowledge sources have been proposed in the past for language modeling. Rosenfeld (1996) uses a maximum entropy model to combine a variety of information sources to improve language modeling on news text and speech. These information sources include complex overlapping n-gram distributions and n-gram caches that aim to capture rare words. The n-gram cache could be considered similar in some ways to our model’s pointer network, where rare or contextually relevant words are stored for later use.

Beyond n-grams, neural sequence models such as recurrent neural networks have been shown to achieve state of the art results (Mikolov et al., 2010). A variety of RNN regularization methods have been explored, including a number of dropout variations (Zaremba et al., 2014; Gal, 2015) which prevent overfitting of complex LSTM language models. Other work has improved language modeling performance by modifying the RNN architecture to better handle increased recurrence depth (Zilly et al., 2016).

In order to increase capacity and minimize the impact of vanishing gradients, some language and translation mod-

	Penn Treebank			WikiText-2			WikiText-103		
	Train	Valid	Test	Train	Valid	Test	Train	Valid	Test
Articles	-	-	-	600	60	60	28,475	60	60
Tokens	929,590	73,761	82,431	2,088,628	217,646	245,569	103,227,021	217,646	245,569
Vocab size	10,000			33,278			267,735		
OoV rate	4.8%			2.6%			0.4%		

Table 1. Statistics of the Penn Treebank, WikiText-2, and WikiText-103. The out of vocabulary (OoV) rate notes what percentage of tokens have been replaced by an *<unk>* token. The token count includes newlines which add to the structure of the WikiText datasets.

els have also added a soft attention or memory component (Bahdanau et al., 2015; Sukhbaatar et al., 2015; Cheng et al., 2016; Kumar et al., 2016; Xiong et al., 2016; Ahn et al., 2016). These mechanisms allow for the retrieval and use of relevant previous hidden states. Soft attention mechanisms need to first encode the relevant word into a state vector and then decode it again, even if the output word is identical to the input word used to compute that hidden state or memory. A drawback to soft attention is that if, for instance, *January* and *March* are both equally attended candidates, the attention mechanism may blend the two vectors, resulting in a context vector closest to *February* (Kadlec et al., 2016). Even with attention, the standard softmax classifier being used in these models often struggles to correctly predict rare or previously unknown words.

Attention-based pointer mechanisms were introduced in Vinyals et al. (2015) where the pointer network is able to select elements from the input as output. In the above example, only *January* or *March* would be available as options, as *February* does not appear in the input. The use of pointer networks have been shown to help with geometric problems (Vinyals et al., 2015), code generation (Ling et al., 2016), summarization (Gu et al., 2016; Gülçehre et al., 2016), question answering (Kadlec et al., 2016). While pointer networks improve performance on rare words and long-term dependencies they are unable to select words that do not exist in the input.

Gülçehre et al. (2016) introduce a pointer softmax model that can generate output from either the vocabulary softmax of an RNN or the location softmax of the pointer network. Not only does this allow for producing OoV words which are not in the input, the pointer softmax model is able to better deal with rare and unknown words than a model only featuring an RNN softmax. Rather than constructing a mixture model as in our work, they use a switching network to decide which component to use. For neural machine translation, the switching network is conditioned on the representation of the context of the source text and the hidden state of the decoder. The pointer network is not used as a source of information for switching network as in our model. The pointer and RNN softmax are scaled

according to the switching network and the word or location with the highest final attention score is selected for output. Although this approach uses both a pointer and RNN component, it is not a mixture model and does not combine the probabilities for a word if it occurs in both the pointer location softmax and the RNN vocabulary softmax. In our model the word probability is a mix of both the RNN and pointer components, allowing for better predictions when the context may be ambiguous.

Extending this concept further, the latent predictor network (Ling et al., 2016) generates an output sequence conditioned on an arbitrary number of base models where each base model may have differing granularity. In their task of code generation, the output could be produced one character at a time using a standard softmax or instead copy entire words from referenced text fields using a pointer network. As opposed to Gülçehre et al. (2016), all states which produce the same output are merged by summing their probabilities. Their model however requires a more complex training process involving the forward-backward algorithm for Semi-Markov models to prevent an exponential explosion in potential paths.

4. WikiText - A Benchmark for Language Modeling

We first describe the most commonly used language modeling dataset and its pre-processing in order to then motivate the need for a new benchmark dataset.

4.1. Penn Treebank

In order to compare our model to the many recent neural language models, we conduct word-level prediction experiments on the Penn Treebank (PTB) dataset (Marcus et al., 1993), pre-processed by Mikolov et al. (2010). The dataset consists of 929k training words, 73k validation words, and 82k test words. As part of the pre-processing performed by Mikolov et al. (2010), words were lower-cased, numbers were replaced with N, newlines were replaced with *<eos>*, and all other punctuation was removed. The vocabulary is the most frequent 10k words with the rest of the tokens be-

ing replaced by an $\langle unk \rangle$ token. For full statistics, refer to Table 1.

4.2. Reasons for a New Dataset

While the processed version of the PTB above has been frequently used for language modeling, it has many limitations. The tokens in PTB are all lower case, stripped of any punctuation, and limited to a vocabulary of only 10k words. These limitations mean that the PTB is unrealistic for real language use, especially when far larger vocabularies with many rare words are involved. Fig. 3 illustrates this using a Zipfian plot over the training partition of the PTB. The curve stops abruptly when hitting the 10k vocabulary. Given that accurately predicting rare words, such as named entities, is an important task for many applications, the lack of a long tail for the vocabulary is problematic.

Other larger scale language modeling datasets exist. Unfortunately, they either have restrictive licensing which prevents widespread use or have randomized sentence ordering (Chelba et al., 2013) which is unrealistic for most language use and prevents the effective learning and evaluation of longer term dependencies. Hence, we constructed a language modeling dataset using text extracted from Wikipedia and will make this available to the community.

4.3. Construction and Pre-processing

We selected articles only fitting the *Good* or *Featured* article criteria specified by editors on Wikipedia. These articles have been reviewed by humans and are considered well written, factually accurate, broad in coverage, neutral in point of view, and stable. This resulted in 23,805 Good articles and 4,790 Featured articles. The text for each article was extracted using the Wikipedia API. Extracting the raw text from Wikipedia mark-up is nontrivial due to the large number of macros in use. These macros are used extensively and include metric conversion, abbreviations, language notation, and date handling.

Once extracted, specific sections which primarily featured lists were removed by default. Other minor bugs, such as sort keys and Edit buttons that leaked in from the HTML, were also removed. Mathematical formulae and $\LaTeX$ code, were replaced with $\langle formula \rangle$ tokens. Normalization and tokenization were performed using the Moses tokenizer (Koehn et al., 2007), slightly augmented to further split numbers ($8,600 \rightarrow 8 @, @ 600$) and with some additional minor fixes. Following Chelba et al. (2013) a vocabulary was constructed by discarding all words with a count below 3. Words outside of the vocabulary were mapped to the $\langle unk \rangle$ token, also a part of the vocabulary.

To ensure the dataset is immediately usable by existing language modeling tools, we have provided the dataset in the

Algorithm 1 Calculate truncated BPTT where every k_1 timesteps we run back propagation for k_2 timesteps

for $t = 1$ **to** $t = T$ **do**

 Run the RNN for one step, computing h_t and z_t

if t **divides** k_1 **then**

 Run BPTT from t down to $t - k_2$

end if

end for

same format and following the same conventions as that of the PTB dataset above.

4.4. Statistics

The full WikiText dataset is over 103 million words in size, a hundred times larger than the PTB. It is also a tenth the size of the One Billion Word Benchmark (Chelba et al., 2013), one of the largest publicly available language modeling benchmarks, whilst consisting of articles that allow for the capture and usage of longer term dependencies as might be found in many real world tasks.

The dataset is available in two different sizes: WikiText-2 and WikiText-103. Both feature punctuation, original casing, a larger vocabulary, and numbers. WikiText-2 is two times the size of the Penn Treebank dataset. WikiText-103 features all extracted articles. Both datasets use the same articles for validation and testing with the only difference being the vocabularies. For full statistics, refer to Table 1.

5. Experiments

5.1. Training Details

As the pointer sentinel mixture model uses the outputs of the RNN from up to L timesteps back, this presents a challenge for training. If we do not regenerate the stale historical outputs of the RNN when we update the gradients, backpropagation through these stale outputs may result in incorrect gradient updates. If we do regenerate all stale outputs of the RNN, the training process is far slower. As we can make no theoretical guarantees on the impact of stale outputs on gradient updates, we opt to regenerate the window of RNN outputs used by the pointer component after each gradient update.

We also use truncated backpropagation through time (BPTT) in a different manner to many other RNN language models. Truncated BPTT allows for practical time-efficient training of RNN models but has fundamental trade-offs that are rarely discussed.

For running truncated BPTT, BPTT is run for k_2 timesteps every k_1 timesteps, as seen in Algorithm 1. For many RNN